

Exercise Sheet 8

Efficient Computing/Parallel Computing 2 in SS 2026

Date of issue: 9.6.2026

Deadline: 16.6.2026

The exercises marked with ★ are relevant for admission to the examination and must be submitted in electronic form in Moodle by the submission deadline.

Exercise 8.1 (*Task Graph Analysis and Critical Path*).

Consider a parallel program that has been decomposed into seven tasks (A through G). The following table lists the computational weights of each task and its immediate predecessors.

Task	Duration (Weight)	Predecessors
A	3	None
B	4	A
C	2	A
D	5	B
E	1	B, C
F	2	C
G	3	D, E, F

- Draw the task dependency graph for these tasks. Represent the tasks as nodes and their dependencies as directed edges. Include the weights in your diagram.
- Perform the forward and backward passes to determine the Earliest Start (ES), Earliest Finish (EF), Latest Start (LS), Latest Finish (LF), and Slack values for each task. Present your results in a table and explicitly identify the critical path.

Exercise 8.2 (*Amdahl's and Gustafson's Laws*).

Consider a parallel computing scenario on a multi-core architecture. We are analyzing the scalability of an application using two distinct performance models: fixed-size speedup (Amdahl's Law) and fixed-time speedup (Gustafson's Law).

- a) Suppose 5% of the total operations in a parallel application must be executed sequentially. According to Amdahl's Law, what is the maximum achievable speedup when executing the program on a system with 64 processors? What is the theoretical maximum speedup if an infinite number of processors were available?
- b) Benchmarking the same application on a 64-processor machine reveals that 8% of the total parallel execution time is spent purely on sequential operations (e.g., synchronization overhead or a root node working while others are idle). Use Gustafson's Law to compute the scaled speedup.
- c) Explain the fundamental conceptual difference between the scaling assumptions made by Amdahl's Law and Gustafson's Law regarding the problem size.

Exercise 8.3 (*Memory-Bounded Speedup: The N-Body Problem*).

While Amdahl's Law analyzes parallel performance assuming a *fixed problem size* (strong scaling) and Gustafson's Law assumes a *fixed execution time* (weak scaling), neither perfectly captures the reality of modern High-Performance Computing (HPC) clusters. In practice, scientists often scale their problems up to the maximum capacity of the available hardware memory.

Sun and Ni introduced the **Memory-Bounded Speedup Model** to address this. The core idea is that when you increase the number of processors p , you also increase the total available system memory. This allows you to run a significantly larger problem. If the sequential fraction f of the workload remains constant, but the parallel portion $1 - f$ scales by a **workload growth factor** $G(p)$ when the memory is fully utilized, the speedup can be much higher than Amdahl's constraint predicts.

Consider the classic naive **N-Body problem**, which simulates the gravitational interactions between N celestial bodies (e.g., stars in a galaxy).

- **Memory Requirement:** Storing the state (position, velocity, mass) of N bodies requires memory proportional to N . Thus, memory scales linearly, $O(N)$.
 - **Computational Workload:** Calculating the gravitational pull of every body on every other body requires evaluating all pairs. Thus, computation scales quadratically, $O(N^2)$.
- a) State the mathematical formula for Sun and Ni's memory-bounded speedup $S_m(p)$. Briefly explain how the workload growth factor $G(p)$ drives this model compared to Amdahl's Law.
 - b) Suppose you scale your N-Body simulation from a single node to a cluster of $p = 64$ nodes. You want to simulate the largest possible universe that fits in the aggregated RAM of the cluster. Determine the workload growth factor $G(64)$ for the parallel fraction of the simulation.
 - c) If the sequential portion of the initial 1-node program is 5% (i.e., $f = 0.05$), calculate the memory-bounded speedup $S_m(64)$ on the 64-node cluster using $G(64)$ calculated above.